

CSV-to-JSON Transformer with Validation

4-Phase Project

Overview

CSV looks like a solved problem until you actually try to parse it yourself. Quoted fields, embedded commas, missing columns, bad types — this is exactly the kind of messy, real-world data handling that separates someone who can write a function from someone who can build a tool other people rely on.

You'll build a small CSV-to-JSON transformer with schema validation, no external CSV libraries allowed, split into four short phases. Timebox each phase and move on even if it isn't perfect.

Each phase also has an optional "JS Concept to Explore" note, tying the code you're already writing to a classic interview topic (closures, arrow functions, higher-order functions, debounce). These aren't extra assignments, they're just a nudge to notice the concept when it naturally shows up in your own code.

General Rules

- Plain JavaScript / Node.js only. No csv-parse, papaparse, or similar libraries — you're building the parser yourself.
 - Write your own test CSV files as you go, especially ones designed to break your own code.
-

Phase 1: Basic Parser

Goal

Turn a simple CSV string into an array of objects, using the header row as keys.

What to Build

```
parseCSV(csvString) -> array of objects
```

Requirements

- First line is the header row and defines the key names for every object.
- Each subsequent line becomes one object in the result array.
- Handle trailing empty lines and extra whitespace around values.

Example

```
const csv = `name,age,email
John,34,john@example.com
Sara,28,sara@example.com`;

parseCSV(csv);
```

```
// [ { name: "John", age: "34", email: "john@example.com" },  
//   { name: "Sara", age: "28", email: "sara@example.com" } ]
```

Deliverable

- parser.js with a working basic parser.
- A sample CSV file and a script that logs the parsed result.

JS Concept to Explore: Closures

Not a strict requirement, just worth trying: instead of a global variable, use a closure to keep track of state while parsing, like the current row number or a running list of warnings. Wrap your parsing logic in an outer function that returns the actual parser, so that state stays private and isn't accessible from outside. This is the same pattern behind things like debounce and memoization, so getting comfortable with it here pays off later. Bonus: in the README, explain in your own words why closures are a common interview topic.

Phase 2: Quoted Fields and Edge Cases

Goal

A naive `split(',')` breaks the moment a field contains a comma inside quotes. Fix that, plus a few other real-world CSV quirks.

Requirements

- Support quoted fields containing commas, e.g. "Doe, John",34,john@x.com should parse as one field "Doe, John", not two.
- Support escaped quotes inside quoted fields, e.g. "She said ""hello"" should become She said "hello".
- Handle empty fields (two commas in a row) as empty strings, not undefined or skipped columns.
- Do not use a single giant regex for the whole line — write a small character-by-character parser so it's actually readable and extendable.

The Hard Part

Get a few real edge cases wrong on purpose first, write a test for them, then fix them. Fields with quotes, commas inside quotes, and escaped quotes together are where most CSV parsers people write from scratch fall apart.

Deliverable

- Updated parser.js handling quoted and escaped fields.
- At least 5 tricky test CSV rows (as a test file or inline test cases) covering the cases above, with expected output for each.

JS Concept to Explore: Arrow Functions vs. Regular Functions

Try writing your character-by-character loop using both a regular function and an arrow function for any helper you extract (like `isQuoteChar` or `isDelimiter`). They'll behave the same here, but it's worth knowing why: arrow functions don't have their own `this`, which matters a lot once you're inside a class or an object method. Not required, but be ready to explain the difference out loud, it comes up constantly in interviews.

Phase 3: Schema Validation

Goal

Raw CSV values are always strings. Add a schema so the parser can convert and validate types, and report problems instead of silently accepting bad data.

What to Build

```
parseCSV(csvString, schema) -> { valid: [...], errors: [...] }
```

Example

```
const schema = { name: "string", age: "number", email: "string" };

parseCSV(csvText, schema);
// {
//   valid: [ { name: "John", age: 34, email: "john@example.com" } ],
//   errors: [ { row: 3, field: "age", message: "Expected number, got \"abc\"" } ]
// }
```

Requirements

- Support at least "string", "number", and "boolean" as schema types.
- Convert values to the correct type when possible (e.g. "34" to 34).
- If a row has a field that fails validation, that row goes into errors with the row number, field name, and a clear message — it should NOT crash the whole parse or silently drop the row.
- Rows that pass every field go into valid.
- Missing columns and extra columns (compared to the schema) should also be reported as errors, not ignored.

Deliverable

- Updated parser.js with schema-based validation.
- A test CSV with a mix of valid rows and rows with type errors, missing fields, and extra fields.

JS Concept to Explore: Higher-Order Functions

Instead of one big if/else chain for checking types, try writing small validator functions per type (validateString, validateNumber, etc.) and a function that takes a type name and returns the right validator, like getValidator("number") returning a function. That's a higher-order function: a function that takes or returns other functions. It's a small change, but it's the same idea behind array methods like map and filter, and it tends to come up in interviews as "write a function that returns a function."

Phase 4: Polish and Report

Goal

Turn this from a working function into something that gives useful feedback, the way a real data-import tool would.

What to Build

```
printReport(result) // human-readable summary to the console
```

Requirements

- Print a short summary: total rows processed, how many valid, how many failed.
- List each error on its own line in a readable format, e.g. Row 3: age — expected number, got "abc".
- Handle a completely empty CSV file and a CSV with only a header row (no data rows) without crashing.
- Add a short README.md explaining how to run it and what the schema format looks like.

Stretch Goal (optional, only if time allows)

Support a simple CLI usage: `node cli.js data.csv schema.json`, reading both files from disk and printing the report.

JS Concept to Explore: Debounce

If you build the CLI stretch goal and have it watch a file for changes (using `fs.watch`) to re-run the parse automatically, you'll notice `fs.watch` can fire multiple times for a single save. Wrap your re-parse call in a debounce function so it only actually runs once the file has stopped changing for a short delay, e.g. 200ms. If you don't build the CLI watcher, just write a 5-line debounce implementation on its own and explain when you'd reach for it in a real app (search input, resize listener, this exact file-watching case).

Final Submission

- Full project folder: `parser.js`, your test CSV files, and `README.md`.
- A few sentences (in the `README` or a separate note) on the trickiest edge case you ran into.

Upload the project (zipped) to the shared Google Drive folder, same as the previous tasks:

<https://drive.google.com/drive/folders/1N-p-yXn3FISrGbnIrtmw170U4QbEFFx?usp=sharing>